CHAPTER 05. 오차역전파법 (BACKPROPAGATION)

5.6절 Affine / Softmax 층 & 5.7절 오차역전파법 구현

행렬 곱의 역전파 메커니즘부터 계층 기반 신경망 완성 및 기울기 검증까지

OVERVIEW

학습 로드맵

5.6.1 - 5.6.2

Affine 계층

행렬의 내적 연산($X \cdot W + B$)과 배치용 Affine 계층의 전파 원리

형상(Shape) 일치의 법칙

5.6.3

Softmax-with-Loss

소프트맥스 함수와 교차 엔트로피 오차가 결합했을 때의 역전파 간소화

말끔한 오차 역전파 ($y - t$)

5.7.1 - 5.7.4

TwoLayerNet & 검증

모듈화된 층을 조립한 신경망 구현과 수치 미분을 통한 기울기 검증

Gradient Check / 빠른 학습

SECTION 5.6.1

Affine 계층: 행렬 곱의 역전파

순전파 연산 수식

$$Y = X \cdot W + B$$

행렬 Shape 일치 조건

$$X\ (2,\ 3) \cdot W\ (3,\ 2) = Y\ (2,\ 2)$$

∂L / ∂X 역전파 공식

$$\partial L/\partial X = \partial L/\partial Y \cdot W^T$$

상류 신호와 가중치의 전치행렬(Transposed Matrix)을 내적

∂L / ∂W 역전파 공식

$$\partial L/\partial W = X^T \cdot \partial L/\partial Y$$

입력 데이터의 전치행렬과 상류 신호를 내적

SECTION 5.6.2

배치용 Affine 계층과 편향(Bias) 역전파

편향(B)의 브로드캐스팅과 역전파

순전파 시 편향 B는 각 데이터에 더해지므로, 역전파 시에는 **모든 데이터의 신호가 편향으로 모여 더해집니다.**

```
self.dB = np.sum(dout, axis=0)
```

```
# Affine 계층 클래스 구현

class Affine:
  def __init__(self, W, b):
    self.W, self.b = W, b
    self.x, self.dW, self.dB = None, None, None

  def forward(self, x):
    self.x = x
    return np.dot(x, self.W) + self.b

  def backward(self, dout):
    dx = np.dot(dout, self.W.T)
    self.dW = np.dot(self.x.T, dout)
    self.dB = np.sum(dout, axis=0)
    return dx
```

SECTION 5.6.3

Softmax-with-Loss 계층의 역전파

Softmax + Cross Entropy Error 조합의 결과

역전파 출력: y - t

(예측값 y에서 정답 레이블 t를 뺀 아주 말끔한 오차 전달)

예시: 정답 t = (0, 1, 0)

예측 $y = (0.3, 0.6, 0.1)$ 일 때
역전파 결과 = $(0.3, -0.4, 0.1)$

의미론적 해석

정답 위치에서는 큰 음수 전달(가중치 증가 유도), 오답 위치에서는 양수 전달(가중치 감소 유도)

SECTION 5.7.1 - 5.7.2

계층 기반 2층 신경망 (TwoLayerNet) 구조

OrderedDict를 활용한 계층 조립

순서가 보장되는 딕셔너리(`OrderedDict`)에 각 층을 순서대로 추가하여, **순전파는 순서대로, 역전파는 반대 순서로 자동 수행**합니다.

신경망 흐름: Affine1 → Relu1 → Affine2 → SoftmaxWithLoss

```
# 역전파(gradient) 메서드 핵심
def gradient(self, x, t):
  # 1. 순전파
  self.loss(x, t)

  # 2. 역전파 (마지막 층부터 시작)
  dout = 1
  dout = self.lastLayer.backward(dout)

  layers = list(self.layers.values())
  layers.reverse()
  for layer in layers:
    dout = layer.backward(dout)
```

SECTION 5.7.3

기울기 검증 (Gradient Check)

수치 미분 (Numerical Diff)

구현이 간단하여 버그가 적지만, 연산 속도가 매우 느림

안전점검 용도

VS

오차역전파법 (Backprop)

연산이 압도적으로 빠르지만, 구현 과정에서 착오가 생기기 쉬움

실제 학습 용도

두 방식으로 구한 기울기 차이의 절대 오차가 10^{-7} 수준으로 극히 작다면 역전파 구현이 올바름을 검증 완료!

SECTION 5.7.4

오차역전파법을 적용한 전체 학습 파이프라인

STEP 1

미니배치 추출

훈련 데이터 무작위 추출

→

STEP 2

기울기 계산

오차역전파법으로 기울기 순간 계산 (`gradient()`)

→

STEP 3

매개변수 갱신

경사하강법으로 가중치 업데이드

5.6 & 5.7절 핵심 요약

구분	순전파 (Forward)	역전파 (Backward)
Affine 계층	`Y = np.dot(X, W) + B`	`dX = np.dot(dout, W.T)` / `dW = np.dot(X.T, dout)`
Softmax-with-Loss	소프트맥스 출력 y 및 손실 L 산출	`dx = (y - t) / batch_size`
기울기 검증	-	수치 미분의 속도 한계를 극복하기 위해 역전파 구현 후 오차 비교 검증

Key Takeaways

신경망을 레고 블록처럼 계층(Layer)으로 모듈화함으로써
아주 깊은 신경망도 효율적이고 유연하게 구축하고 오차역전파를 수행할 수 있습니다.

감사합니다